

UNIVERSITY OF LAY ADVENTISTS OF KIGALI

Faculty of Computing and Information Sciences

FINAL EXAMINATION REPORT

DPR400210 – Database Programming with Oracle Database

AI-Assisted Fraud Detection System for Mobile Payments

Prepared by: Dianna

Student ID: 26585/2024 | Academic Year 2025–2026

Database/User: Dianna_26585_2024_FraudDetection_DB

Instructor: Eric Maniraguha

1. Problem Statement

Mobile money platforms process massive transaction volumes daily. Manual review cannot keep pace with fraud patterns such as SIM-swap fraud, agent collusion, and rapid micro-transaction “smurfing.” A relational database system is needed to flag suspicious activity, score risk, and route cases for investigation.

Context of use

Deployed within a mobile money operator's Risk & Compliance department, integrated with transaction processing and agent networks.

Target users

- Fraud Analysts
- Risk Managers
- Compliance Officers
- Database/System Administrators

Objectives

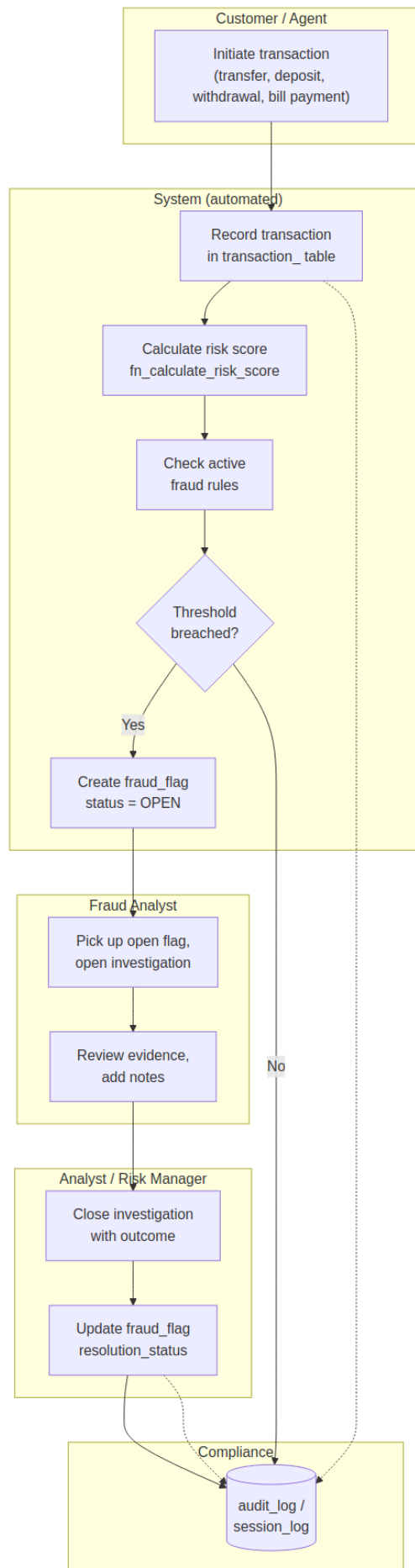
- Model customers, agents, devices, accounts, and transactions relationally
- Score transactions with rule-based and statistical anomaly checks
- Flag, queue, and track investigation of suspicious activity
- Enforce business rules via triggers (weekday/holiday DML restrictions)
- Maintain a full audit trail of account and rule changes
- Deliver an analytics dashboard on fraud trends

Expected benefits

- Faster fraud detection and response time
- Reduced financial losses from fraud
- Audit-ready compliance trail
- Data-driven insight into fraud patterns

2. Business Process Modeling

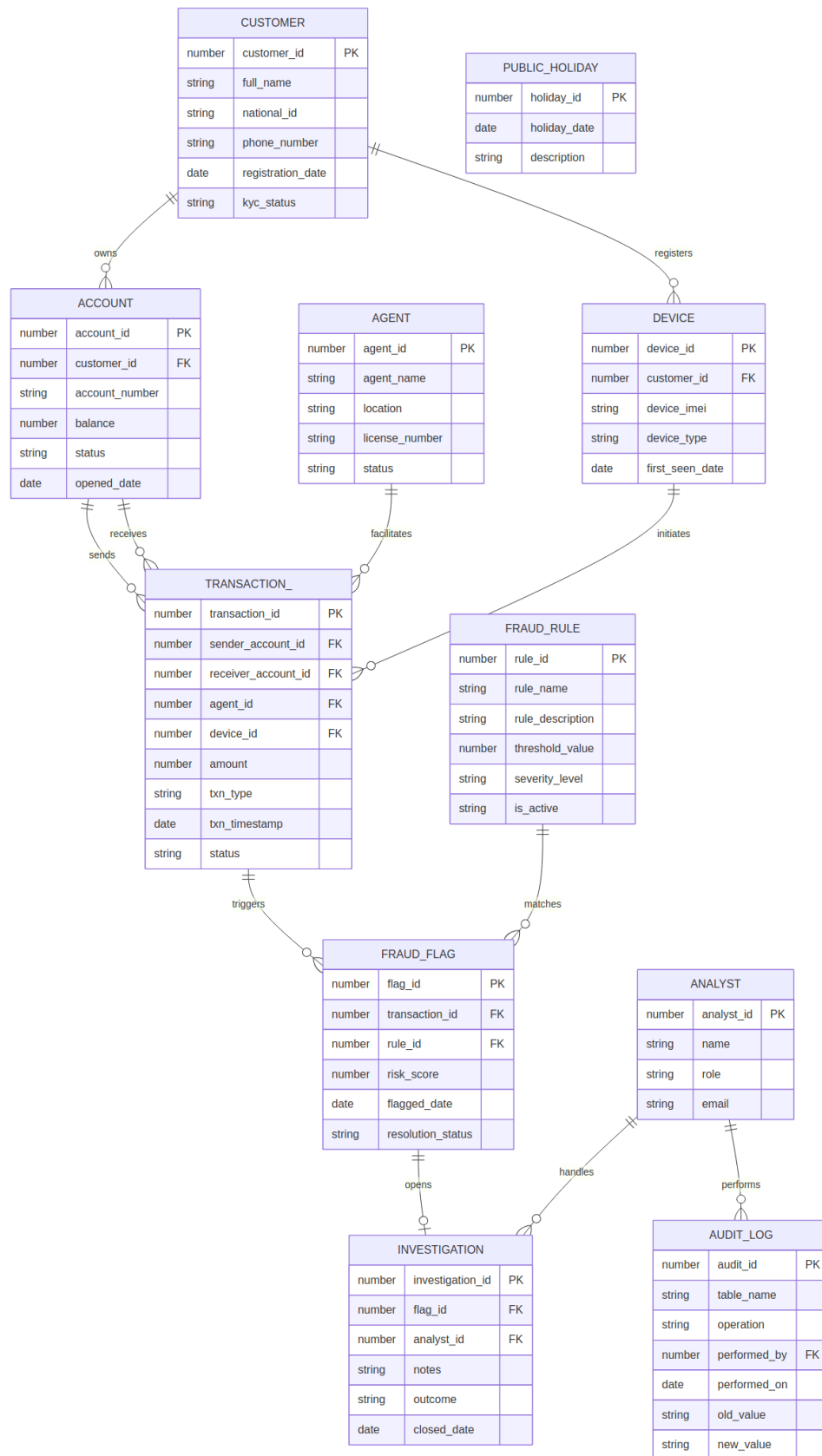
The diagram below models the full fraud-detection lifecycle across five swimlanes: Customer/Agent, System, Fraud Analyst, Analyst/Risk Manager, and Compliance.



Workflow summary: a customer initiates a transaction (optionally via an agent); the system records it, calculates a risk score, and checks it against active fraud rules; if a threshold is breached, a fraud flag is created and an analyst opens an investigation; the analyst or risk manager closes the case with an outcome; every step is written to the audit log for compliance.

3. Logical Database Design (3NF)

The schema below covers 11 entities: CUSTOMER, DEVICE, AGENT, ACCOUNT, TRANSACTION_, FRAUD_RULE, FRAUD_FLAG, ANALYST, INVESTIGATION, PUBLIC_HOLIDAY, and AUDIT_LOG.



Normalization justification

- 1NF: every attribute is atomic — no repeating groups or comma-separated values.
- 2NF: every table uses a single-column surrogate primary key, so no partial dependency is possible.
- 3NF: no transitive dependencies — e.g. agent_name and location live only in AGENT, not repeated on every transaction row.

4. Database Creation

A dedicated tablespace (fraud_db_ts) and schema user (Dianna_26585_2024_FraudDetection_DB) were created with the minimum privileges needed: CREATE SESSION, TABLE, VIEW, SEQUENCE, PROCEDURE, TRIGGER, and SYNONYM, plus unlimited quota on the tablespace. A read-only fraud_analyst_role was also created for reporting/dashboard access without granting schema-owner rights.

5. Table Implementation

All 11 core tables were created with primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK constraints. Key examples:

Table	Key constraint	Why
customer	UNIQUE(national_id), UNIQUE(phone_number)	Prevents duplicate registrations
account	CHECK(balance >= 0)	Blocks negative balances at the DB level
transaction_	CHECK(sender_account_id <> receiver_account_id)	Blocks self-transfers
fraud_flag	CHECK(risk_score BETWEEN 0 AND 100)	Keeps scores within a valid range
investigation	UNIQUE(flag_id)	One investigation per fraud flag

Meaningful sample data was inserted, including one deliberately large transaction (700,000 RWF) that breaches the LARGE_SINGLE_TXN rule — used throughout later phases to demonstrate the fraud-scoring and investigation workflow live.

6. PL/SQL Programming

fn_calculate_risk_score(transaction_id) — function

Computes a 0–100 risk score from three signals: large transaction amount, a brand-new device, and rapid repeated transactions from the same device within 10 minutes.

pr_register_transaction(...) — procedure

The single entry point for creating a transaction: validates business rules with named exceptions, inserts the row, scores it, and loops through active fraud rules with a cursor to automatically create a fraud_flag when a threshold is breached. Commits on success, rolls back on error.

fraud_pkg — package

Bundles register_transaction, open_investigation, close_investigation, and get_open_flag_count behind one public interface, so calling code only needs to know the package.

Demo test calls (see 06_plsql/phase6_plsql.sql) confirm: a normal transaction registers with no flag, a large transaction triggers a flag with an elevated risk score, an invalid transaction (same sender/receiver) raises the expected custom exception, and the open-flag count decreases correctly after an investigation is closed as a false positive.

7. Triggers, Auditing & Security

Mandatory business rule

trg_block_weekday_holiday_dml is a compound trigger that blocks INSERT/UPDATE/DELETE on transaction_ during weekdays (Monday–Friday) and public holidays (checked against the public_holiday reference table), as required by the brief.

Audit system

trg_audit_account and trg_audit_fraud_flag write every INSERT/UPDATE/DELETE to audit_log, capturing the operation and old/new values, without ever blocking the underlying business transaction if the audit insert itself fails.

User activity tracking

trg_track_logon fires AFTER LOGON ON SCHEMA and records every connection (username, OS user, host) into session_log.

Security restriction

trg_protect_customer_delete adds a defense-in-depth layer: even a user with DELETE privilege on customer cannot delete a row unless they are the schema owner.

8. Conclusion

This project delivers a complete Oracle Database solution for mobile money fraud detection: a normalized 3NF schema, enforced business rules, automated risk scoring in PL/SQL, compliance-grade auditing, and an analytics layer for ongoing monitoring. The design demonstrates technical mastery of SQL and PL/SQL, sound data modeling, and a real-world approach to a problem mobile money operators face daily.